

CS 6888: Program Analysis and its Applications

Assignment 2: Symbolic Execution

Due: February 20th at 5 PM
Worth 10 points

1 Learning Objective

To consolidate your understanding of, and obtain hands-on experience with **symbolic execution**, including:

- how it works
- its applications in test generation
- its strengths and weaknesses

2 Individual Assignment

This is an individual assignment. You are not to share your setup, strategies, results, analysis, code, or any other aspect of the assignment with your classmates.

3 Preparation

3.1 Environment

This assignment will use KLEE, a symbolic execution engine for test generation. We'll use a separate docker image for this assignment, which you can download and run with:

```
docker pull klee/klee:3.0
docker run -v $(pwd)/files/hw2:/home/klee/hw2 --rm -ti --ulimit="stack=-1:-1" klee/klee:3.0
```

(Arm Macs should add the `--platform linux/amd64` flag to `docker run`.)

Note: run this from the root of your fork and it will map your `files/hw2` directory to `~/hw2` in the container (it is important that that is the path inside the container for grading). It should work on Linux, MacOS, and BSD hosts.

It may also work on WSL. For windows directly, you'll need to replace `$(pwd)` with the absolute path to the directory. (`%cd%` for command prompt and `${PWD}.Path` for powershell may also work, but I haven't tested them.)

For people who cannot use docker on their own machine, we have installed KLEE 3.2 in apptainer on portal.cs.virginia.edu. To use it you need to run the following commands:

```
module load apptainer klee
apptainer run $CONTAINERDIR/klee-3.2.sif
```

Whether you use KLEE 3.0 on docker or KLEE 3.2 on portal, the assignment is the same as described below.

3.2 Practice

Before you start the assignment, you should familiarize yourself with KLEE:

3.2.1 Documentation

In addition to the main `klee` executable, there are two useful utilities that generate human-readable versions of KLEE's outputs, `ktest-tool` and `klee-stats`.

- klee: <https://klee.github.io/docs/>
- ktest-tool: <https://klee.github.io/docs/tools/#ktest-tool>
- klee-stats: <https://klee.github.io/docs/tools/#klee-stats>

3.2.2 KLEE Tutorials

- <https://klee.github.io/tutorials/testing-regex/>
- <https://klee.github.io/tutorials/using-symbolic/>
- <https://feliam.wordpress.com/2010/10/07/the-symbolic-maze/>

3.2.3 Understanding KLEE's Options

- <https://klee.github.io/docs/options/>

3.2.4 Understanding KLEE's Outputs

- <http://ccadar.blogspot.com/2020/07/measuring-coverage-achieved-by-symbolic.html>

4 Tasks

4.1 Develop a benchmark of programs.

Based on your understanding of symbolic and concolic execution and of KLEE (as per its documentation and tutorials), you are to build a benchmark of small programs that showcase KLEE's strengths and weaknesses.

More specifically, you are to develop:

1. 5 programs with diverse and representative features for which KLEE can explore all feasible paths (gets 100% branch coverage)
2. 5 programs with diverse and representative features for which KLEE struggles to explore all feasible paths
Struggle in this context means that KLEE is unable to generate a test that traverses that path within 30 minutes. There are two ways to achieve this:
 - (a) KLEE is still running at 1800 seconds
If this is the case, you should include the `--max-time=1800` argument in your `run_all.sh`.
 - (b) KLEE terminates and reports incomplete paths/branch coverage
If this is the case, you should be very sure that the uncovered parts are actually what you're testing and show limitations of symbolic execution. You should also explain one of the missed paths in your report. (You can do this by looking at the individual test files using `ktest-tool`.)

The programs in the benchmark must be:

1. Original
Be developed by you and only you for this assignment
2. Distinct
Be distinct from the ones included in the KLEE demos and tutorials or papers using KLEE
3. Concise
Be smaller than 50 lines of code (when you run `wc -l <file>` on it)
4. Targeted
Capture a single specific program feature. The target program feature must be used in the name of the program file. For example, if your program is meant to showcase how KLEE deals with an iteration through a loop, you could call the program `loop-iteration.c`.
5. Diverse
Be diverse. You want each program to show a unique strength or a weakness in KLEE that is not shown by the other programs in the benchmark. So, for example, showing how KLEE handles a pointer in a program could be interesting, but also showing a program with multiple pointers would not add much in terms of diversity. Similarly, having a program with the trigonometric function 'sin' could be interesting, but having another one with 'tan' would not add

much in terms of showcasing what KLEE can and cannot handle. You must aim to have a suite that explores different aspects of the language to show that diversity.

6. Representative

Be representative of program features used in practice. We are not looking for corner cases in a programming language that are not handled by KLEE, but rather common coding constructs that we expect for it to handle.

Note: KLEE doesn't support floating point operations; while there is a fork that does, that fork is not allowed for the homework. Instead, you can find fixed-point implementations of most functions, e.g. <https://www.nullhardware.com/blog/fixed-point-sine-and-cosine-for-embedded-systems/>, if you wish to use them. (If you use any external resources, including this one, you must say how/why/where you used it and cite it (in addition to the normal explanations required for the HW).)

4.2 Write a report.

The report must be under 3 pages in ieee-conference format¹ and include three sections with the following contents:

1. Program benchmark design considerations.

For each of the 10 programs, explain the program feature considered and justify why it was selected in terms of diversity and representativeness (i.e. why it meets the above criteria).

2. Coverage Report

Create a table showing the number of tests and branch coverage achieved by KLEE for each program in the benchmark. Each program is a row in the table, the columns are # tests and coverage. Make sure that each program clearly maps to one of the selected features discussed in the previous item.

3. Analysis

- (a) Highlight and explain key observations revealed by your benchmark in terms of programming constructs that KLEE handles either particularly well or badly.
- (b) Were you surprised by KLEE's handling or mishandling of any construct? Explain specifically why or why not.
- (c) For the constructs that are not handled well, conjecture why that is the case based on your understanding of symbolic execution.
- (d) For each program that struggles, theorize whether concolic execution would be able to evaluate the program effectively. Explain why/why not.

5 Submission

5.1 Report

Submit your report through Canvas.

5.2 Code

Your repo will be cloned at the due date. Be sure that you have committed and pushed your code at that time.

¹Templates are available at <https://www.ieee.org/conferences/publishing/templates.html> or <https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhnscsfqn>.